L-Int: Von Ausdrücken zum Interpreter

Carsten Gips (HSBI)

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Von AST zu Werten: Idee

- Gegeben: Lexer/Parser haben einen AST aufgebaut
- Gesucht: `eval(expr)` berechnet einen **Wert**

- Beispiele:
  - `eval( IntLiteral(1) )` → `1`
  - `eval( AddExpr(IntLiteral(1), IntLiteral(2)) )` → `3`

- Frage:
  - Wie repräsentieren wir solche Werte sauber im Code?

Value-Hierarchie: Motivation

```
Expr = IntLiteral(Int) | AddExpr(Expr, Expr) | SubExpr(Expr, Expr)
Lint = Expr
```

- L-Int: Nur ganze Zahlen
- Später:
  - Booleans
  - Funktionen
  - Objekte

→ `Value` als gemeinsamer Basistyp für Laufzeitwerte

Value-Hierarchie in Java

```java
public interface Value {

    static int asInt(Value v) {
        if (v instanceof IntVal(int val)) return val;
        throw new RuntimeException("number expected");
    }
}


public record IntVal(int v) implements Value {}
```

- Klar getrennt von `Expr` (Syntax vs. Laufzeitwerte)

Tree-Walking-Interpreter: Grundidee

- Funktion: `Value eval(Expr expr)`

- Idee:
  - Rekursiv über den AST laufen
  - Pro Knotentyp die passende Operation ausführen:
    - Literal (Integer): Als Integer-Wert zurückgeben
    - Ausdruck (Addition, Subtraktion): linke und rechte Seite zu Integer-Werten evaluieren, die Operation ausführen, Ergebnis als Integer-Wert zurückgeben

Interpreter für L-Int-Expressions in Java (Pattern Matching)

```java
public class InterpreterLint {

    public Value eval(Expr expr) {
        return switch (expr) {
            case IntLiteral(var v) -> new IntVal(v);

            case AddExpr(var l, var r) -> {
                int a = Value.asInt(eval(l));
                int b = Value.asInt(eval(r));
                yield new IntVal(a + b);
            }

            // TODO: SubExpr

            default -> throw new RuntimeException("unexpected expression: " + expr);
        };
    }
}
```

Interpreter in Aktion

```
// (- (+ 1 2) 3)
Expr expr =
    new SubExpr(
        new AddExpr(new IntLiteral(1), new IntLiteral(2)),
        new IntLiteral(3)
    );

var interp = new InterpreterLint();
Value result = interp.eval(expr);
// result: IntVal(0)
```

Wrap-up

- Interpreter simulieren die Programmausführung
  - Code ausführen: Read-Eval-Loop (REPL)
- Tree-Walking-Interpreter traversieren den AST
- Auswertung der Knoten mit `eval(Expr e)`



Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

Last modified: *136fe43 2026-11-07 lexpr: improve slides*